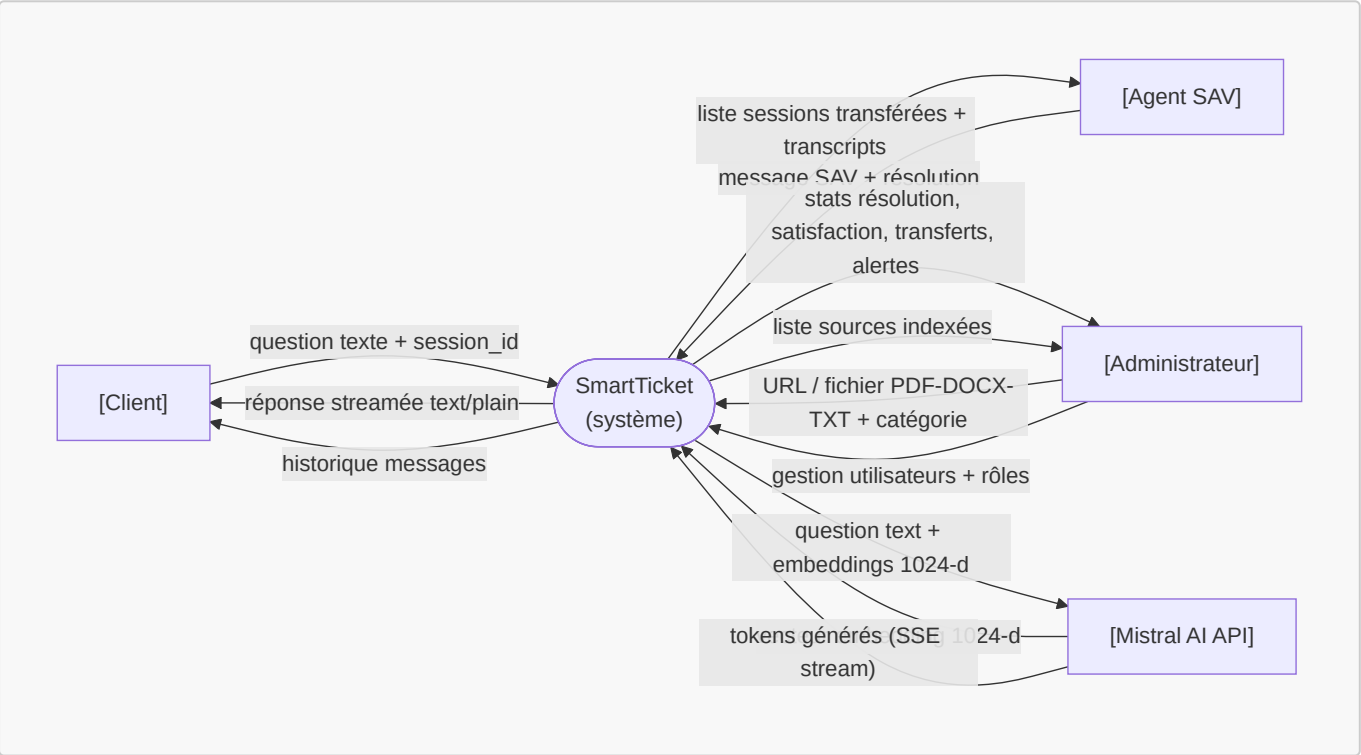
Livrable 2 — Diagramme de flux de données (DFD)

SmartTicket — Pipeline RAG et gestion de tickets

> Les diagrammes sont construits à partir de l'analyse du code source (`backend/routers/`, `backend/mistral_client.py`, `backend/dependencies.py`). Chaque flux est étiqueté avec la donnée réelle qui circule.

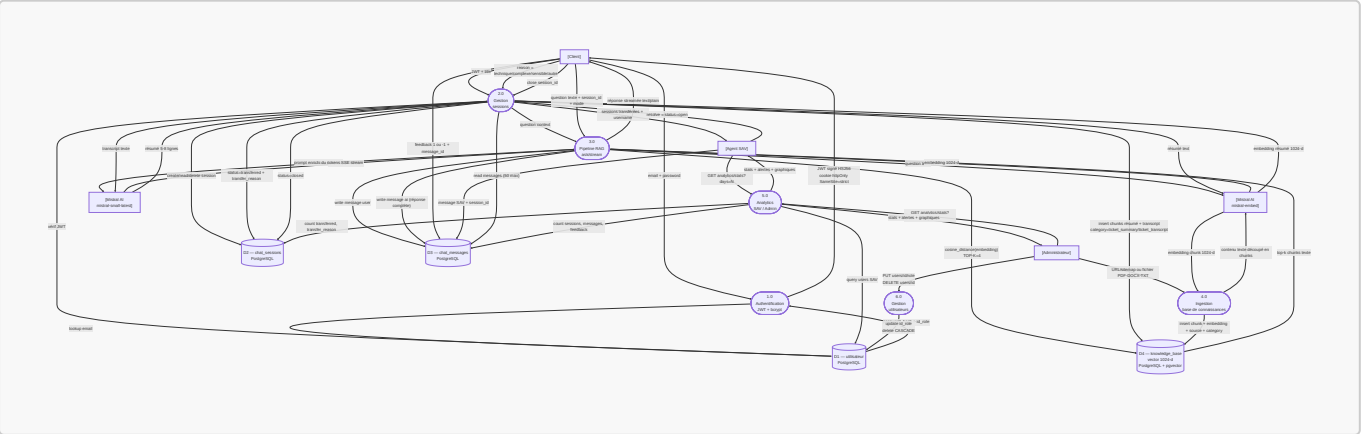
2.1 DFD Niveau 0 — Contexte (vue boîte noire)

SmartTicket est représenté comme un système unique. Les entités externes interagissent avec lui via leurs flux principaux.



2.2 DFD Niveau 1 — Décomposition des processus internes

Les processus numérotés correspondent aux modules effectivement présents dans le code.



Légende des flux

Flux	Type de donnée
`email + password`	Credentials JSON (Pydantic UserLogin)
`JWT signé HS256`	Token JWT dans cookie httpOnly (max_age=3600s)
`question texte + session_id`	JSON `{question, session_id, mode}`
`embedding 1024-d`	Vecteur float32[1024] (mistral-embed)
`cosine_distance TOP-K=4`	Requête pgvector `ORDER BY embedding <=> query_vec LIMIT 4`
`top-k chunks texte`	Liste de strings (contenu des lignes knowledge_base)
`prompt enrichi du contexte`	String concaténant contexte KB + question (max 3000 chars)
`tokens SSE stream`	`text/plain` chunked (Server-Sent stream Mistral)
`feedback 1 ou -1`	PATCH `{feedback: 1
`status=transferred`	UPDATE `chat_sessions.status + transfer_reason`
`insert chunks résumé`	Entrées `knowledge_base` category=`ticket_summary`/`ticket_transcript`
`stats + alertes`	JSON avec `ai_resolution_rate`, `satisfaction_score`, `transfer_reasons`, `alerts[]`

Datastores détaillés

Datastore	Table SQL	Colonnes clés	Usage
D1	`utilisateur`	id, email, password_hash, id_role	Auth, RBAC, liste agents SAV
D2	`chat_sessions`	id, id_utilisateur, status, transfer_reason	Gestion cycle de vie des tickets
D3	`chat_messages`	id, id_session, type_envoyeur, contenu, feedback	Historique + analytics feedback
D4	`knowledge_base`	id, contenu, embedding vector(1024), category, source	Recherche vectorielle RAG (index HNSW cosine)

> **Note sur Redis** : un datastore D5 (Redis) était prévu pour la mise en cache des sessions/résultats LLM mais n'est pas connecté dans le code actuel. L'état des jobs d'ingestion est stocké en mémoire vive (`INGEST_JOBS` dict dans `dependencies.py:27`).